

Section 1: Multiple Choice Questions (MCQs)

1. **What is the primary programming language used in the OpenAI Agents SDK?**

2. A) JavaScript

3. B) Go

4. C) Python 

5. D) Rust

6. **What is the purpose of a "Tool" in the OpenAI Agents SDK?**

7. A) To decorate functions for GUI support

8. B) To define functions an agent can call 

9. C) To manage prompts

10. D) To validate user input only

11. **What is the role of `` in the OpenAI SDK?**

12. A) Creates an async context

13. B) Runs the agent loop asynchronously

14. C) Executes the agent loop synchronously 

15. D) Compiles Pydantic models

16. **Which model type is typically used in OpenAI Agents SDK for LLM-based orchestration?**

17. A) Decision Trees

18. B) DistilBERT

19. C) Chat Completions 

20. D) Reinforcement Learners

21. **What Python package is used for input/output schema validation in tools?**

22. A) marshmallow

23. B) dataclasses

24. C) pydantic ✓
25. D) jsonschema
-

Section 2: True/False

1. The OpenAI Agents SDK can only be used with OpenAI-hosted LLMs. — **False**
 2. The `context` object is passed to tools to share information across the agent loop. — **True**
 3. Markdown images require the format: `[alt text](url)`. — **False** (Correct: `![alt text](url)`)
 4. You can use `async def` with tools in the OpenAI SDK. — **True**
 5. The agent loop automatically retries failed tool calls by default. — **False**
-

Section 3: Short Answers

1. **What are "handoffs" in the context of multi-agent collaboration?**

A primitive that allows transferring tasks or context between agents.

1. **List three core components of an OpenAI Agent.**

Agent, Tool(s), and Runner.

1. **What does "Chain-of-Thought" prompting refer to?**

A prompting method that guides the LLM to reason step by step.

1. **Why is context persistence important in agent design?**

To retain relevant information and avoid re-computation or loss of dialogue memory.

1. **What's one benefit of using `` in tool definitions?**

Automatic input validation and structured schema.

vs Section 4: Fill in the Blanks

1. `@tool` decorator is used to register _____ with an agent. — **functions**
2. In Markdown, to create a heading you use the _____ symbol. — **#**

3. To generate structured outputs, the agent uses _____ models in Python. — **Pydantic**
4. A guardrail in agentic AI ensures _____ behavior and avoids invalid responses. — **safe**
5. `top_p` is used in LLMs to apply _____ sampling. — **nucleus**

Section 5: Code Snippets (Identify Errors or Output)

1. What is wrong with this tool definition?

```
@tool
def get_weather(city: str):
    return f"Weather in {city} is sunny."
```

Answer: Missing Pydantic model for input typing; should wrap `city` inside a BaseModel.

1. Fix the Markdown syntax:

```
[Google.com](https://www.google.com)
```

Answer: `[Google](https://www.google.com)`

1. Predict the output:

```
from pydantic import BaseModel

class Input(BaseModel):
    number: int

def square(input: Input):
    return input.number ** 2

square(Input(number=3))
```

Answer: `9`

1. What tool call structure will this function expect?

```
@tool
class AddInput(BaseModel):
    x: int
    y: int
```

```
def add_numbers(input: AddInput):  
    return input.x + input.y
```

Answer:

```
{  
  "x": 3,  
  "y": 4  
}
```

1. Which method do you use to run agents in sync mode?

Answer: `Runner.run_sync()`

Section 6: Prompt Engineering

1. What is the goal of prompt engineering in agent design?

To guide the agent's behavior and outputs toward desired responses using structured natural language prompts.

1. Describe "Tree-of-Thought" prompting.

A method that branches out multiple reasoning paths to explore different solutions before selecting the best one.

1. Give one example of persona prompting.

"You are a helpful and concise medical assistant."

1. Why should we avoid ambiguous phrasing in prompts?

Ambiguity leads to inconsistent or irrelevant LLM outputs.

1. What technique helps the LLM break down complex reasoning into simpler steps?

Chain-of-Thought

Section 7: Practical Understanding

1. What are two main parameters controlling randomness in LLM generation?

`temperature` and `top_p`

1. Why is using `` useful in tool functions?

Allows for concurrent execution of I/O-bound tasks (e.g., web requests).

1. In multi-agent systems, how is task delegation usually achieved?

Via handoffs and dynamic routing logic.

1. Which component is responsible for the main decision-making in an agentic app?

The Agent

1. What's the importance of tool descriptions in the Agents SDK?

Guides the model in selecting the correct tool during reasoning.

 Section 8: Markdown Cheat Sheet (Syntax-Based)

1. Code block: write syntax to show `print("Hello")` as Python code.

```
print("Hello")
```

37. Write a Markdown image syntax with URL ``https://example.com/cat.png``
> ``![A cute cat](https://example.com/cat.png)``

38. Make a numbered list with three items.
>

1. Apple
2. Banana
3. Cherry

39. Create a hyperlink to OpenAI.
> ``[OpenAI](https://openai.com)``

40. How do you bold text in Markdown?
> ``**text**``

 Section 9: Conceptual Deep-Dive

41. **What is the difference between ``Agent.run()`` and ``Runner.run_sync()``?**
> ``Agent.run()`` handles a single request/turn, while ``Runner.run_sync()`` runs

the full conversational loop.

42. **What is meant by "agent orchestration"?**

> Coordinating multiple agents and tools to work together on tasks dynamically.

43. **Name a key benefit of using Agents SDK for tool calling.**

> Automatic structured input parsing and error handling via Pydantic.

44. **How does Agents SDK handle tool errors?**

> It allows defining custom error messages and retries in tool logic.

45. **What's the purpose of tool annotations (e.g., '@tool')?**

> To register the function or class with the agent as an invocable tool.

Section 10: Knowledge Recall

46. **What organization develops the Agents SDK?**

> OpenAI

47. **What Python library is required for defining agent tools?**

> Pydantic

48. **Name one real-world application of multi-agent collaboration.**

> Customer support automation (e.g., different agents for billing, tech, etc.)

49. **Which Agent SDK method allows streaming responses to users?**

> `run_stream()`

50. **Mention one type of guardrail in agent design.**

> Content filtering or schema validation